The brief

Low-Code Streamlit EventOps

An independently operating event-operations companion built with **Streamlit, n8n, FastEmbed and Gemini**. Retrieval and model interpretation support the workflow; deterministic verification owns the result.

PYTHON / STREAMLIT WORKFLOW AUTOMATION EVIDENCE-LED AI

Executive perspective

I built this companion to test a different delivery model from the original React/FastAPI EventOps implementation. Streamlit provides an efficient internal workflow interface; n8n executes visible source-ingestion and verification stages; focused Python modules preserve authoritative event and participant rules. The system retains source evidence, manual review decisions and run history without treating an LLM response as a verification verdict.

Operational scope

Ask and inspect

Semantic event, directory and policy lookup, follow-up context, structured evidence and guarded Gemini explanations.

Run and reconcile

Authenticated n8n workflows read independent sources, normalize records, preserve conflicts and produce verified/review/rejected/incomplete outcomes.

Review and export

Reviewer reasons, notes and proposed actions remain separate from authoritative status. Saved runs and protected CSV exports can be reopened.

Local-first operations

Persistent local databases and container workflows are tested. Public hosting is prepared but has not occurred.

A low-code interface.
Explicit engineering boundaries.

Streamlit shortens UI delivery, but it does not remove the need for durable state, validated sources, bounded model calls or independently testable business rules.

Architecture responsibilities

Streamlit interface

Questions, search results, scenario/run controls, review actions, history, source inspection and CSV downloads.

FastEmbed / NumPy

Three structured documents produce 19 stable 384-dimensional vectors; scoped cosine ranking and explicit clarification support retrieval.

n8n verification workflow

A 31-node graph executes source reads, stage checkpoints, aggregation, normalization, decision branches and persistence.

Python processing

Deterministic parsing, duplicate/conflict preservation, reference matching, provenance, and distinct record statuses.

Gemini investigation

An 11-node n8n AI Agent uses three read-only tools; actual observations and evidence citations are validated before display.

Persistence / lifecycle

SQLAlchemy data, revision checks, owner-scoped reviews, atomic replay claims and conservative reconciliation of uncertain execution.

Three implementation rules

Sources remain distinct

Synthetic Calendar/Web HTTP is identified as controlled; Mailtrap messages are genuine read-only sandbox fetches.

Models remain advisory

Generated findings cannot change directory or event verification truth. Unsupported citations are rejected.

Uncertainty is retained

A stale n8n checkpoint requires investigation, not an invented success or automatic replay.

Measured locally.
Not presented as deployed.

The latest Iteration 003 release evidence confirms the deterministic system, real workflow execution and recovery rehearsal. Model activity in Iteration 002 is historical evidence; Gemini was deliberately disabled during Iteration 003 release acceptance.

Observed verification

147 regression tests

Passed on macOS and disposable Linux ARM64. The release GitHub Actions Python and container jobs also passed.

30 / 30 curated RAG cases

Behavioral checks passed against 19 records; 25 ranked cases reached Recall@5=1.0 and MRR=1.0 on this small corpus.

Actual n8n processing

The main synthetic scenario produced 8 merged records: 3 verified, 2 review, 2 rejected, 1 incomplete; 2 duplicate inputs merged.

Recovery and storage

Isolated interruption, restart preservation and a custom PostgreSQL archive restored into a new empty database were tested.

Deployment position and limitations

A protected six-service deployment topology, Caddy gateway, separated credentials, workflow publication, health checks, backup/restore and release runbooks are prepared. **No public URL, hosted acceptance, Neon connection, live TLS or approved resource provisioning has been verified.** The release browser tool blocked its localhost target; the native Streamlit AppTest passed, but a real release-browser acceptance remains outstanding. The planned $0 hosting path depends on provider eligibility and actual capacity.

Project distinction

The Streamlit/n8n companion is a separate implementation from the custom React/FastAPI/LangGraph EventOps application. It demonstrates lower-code interface delivery and genuine workflow orchestration while retaining independently tested Python rules, evidence boundaries and realistic deployment constraints.

SOURCE / github.com/keenosmith-del/eventops-intelligence-streamlit

EVIDENCE / docs/iterations/003/TEST_RESULTS.md · RELEASE_READINESS.md